DevOps Laboratory

Assignment 2

Continuous Integration Using GitHub Actions

Student Name	YUVRAJ SHEMBALE
Roll No / PRN	123B1D045
Date of Submission	

1. Aim

To design and implement an automated Continuous Integration (CI) workflow using GitHub Actions that builds, tests, and validates code changes whenever they are pushed to a repository.

2. Introduction

Continuous Integration (CI) is a DevOps practice in which developers frequently integrate code changes into a shared repository, with each integration automatically verified by a build and automated tests. This early detection of integration problems reduces the cost and effort of fixing bugs later in the development cycle.

GitHub Actions is a native CI/CD platform built directly into GitHub repositories. It allows developers to define automated workflows using simple YAML configuration files stored under .github/workflows/. These workflows can be triggered by events such as a push, a pull request, or a scheduled timer, and can execute jobs consisting of multiple ordered steps on hosted or self-hosted runners.

3. Technologies / Tools Used

Technology	Purpose / Role
GitHub Actions	Native CI/CD platform used to define and run automated workflows
YAML	Configuration language used to write the workflow definition file
GitHub Repository	Hosts the source code and triggers workflow events
Ubuntu Runner (ubuntu-latest)	Hosted virtual machine that executes the workflow jobs
Node.js / Python (application runtime)	Runtime environment used to build and test the sample application

4. Project / Repository Structure

```
ci-actions-demo/
■■■ .github/
■   ■■■ workflows/
■       ■■■ ci.yml            # CI workflow definition
■■■ src/
■   ■■■ app.js                # sample application code
■■■ test/
■   ■■■ app.test.js           # sample test file
■■■ package.json
■■■ README.md
```

5. Procedure

1. Create a GitHub repository containing a simple application (e.g., a small Node.js or Python project).

2. Create the workflow directory .github/workflows/ and add a workflow file such as ci.yml.

3. Define the workflow to trigger on push (and optionally pull_request) events to the main branch.

4. Configure a job that runs on ubuntu-latest, checks out the repository code using actions/checkout, and sets up the required runtime environment.

5. Add steps to install application dependencies and execute a build or test command.

6. Push the workflow file to GitHub and observe the workflow execute automatically under the Actions tab.
7. Introduce an intentional error (e.g., a failing test or a syntax error) and observe the workflow report a failure.
8. Correct the error, push the fix, and verify that the workflow now completes successfully.

6. Code / Configuration Snippets

6.1 Basic CI Workflow (.github/workflows/ci.yml)

```yaml
name: CI Workflow

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

6.2 Sample Application File (src/app.js)

```javascript
function add(a, b) {
    return a + b;
}

module.exports = { add };
```

6.3 Sample Test File (test/app.test.js)

```javascript
const { add } = require('../src/app');

test('adds two numbers correctly', () => {
    expect(add(2, 3)).toBe(5);
});
```

6.4 Intentional Failure Example (for demonstration)

```javascript
// Introducing a bug to observe a failed workflow run:
function add(a, b) {
    return a - b; // incorrect operator
}
```

7. Output / Screenshots

The following screenshots demonstrate successful execution of each major step. Replace each placeholder with the corresponding screenshot captured during the practical.

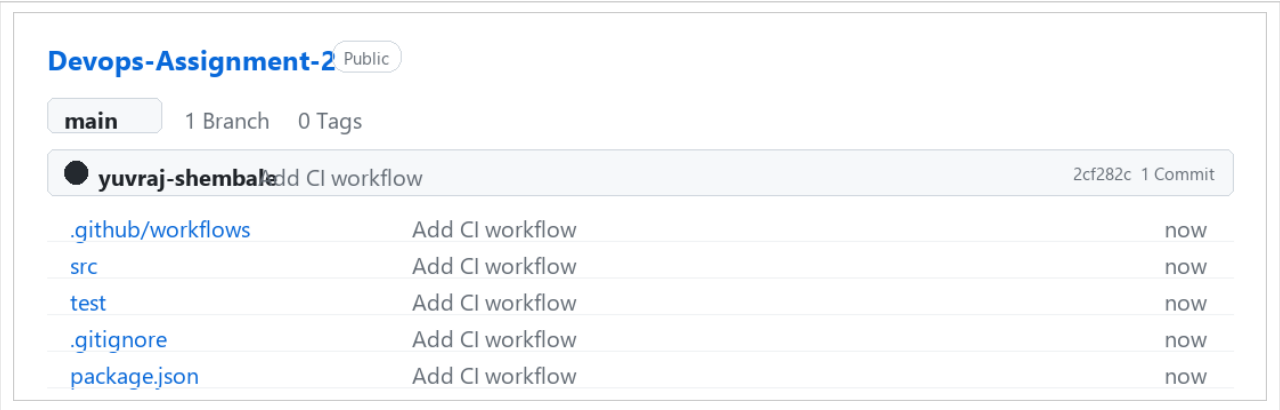


Fig 1: GitHub repository with .github/workflows/ci.yml committed

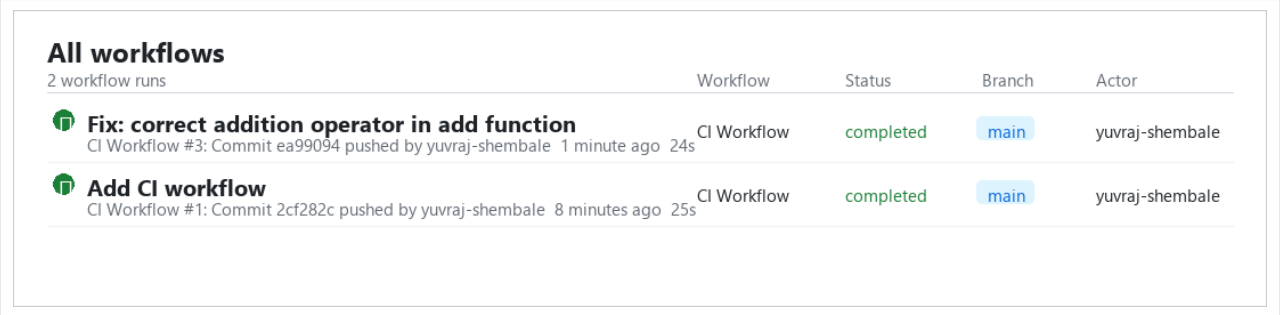


Fig 2: Actions tab showing the workflow triggered on push

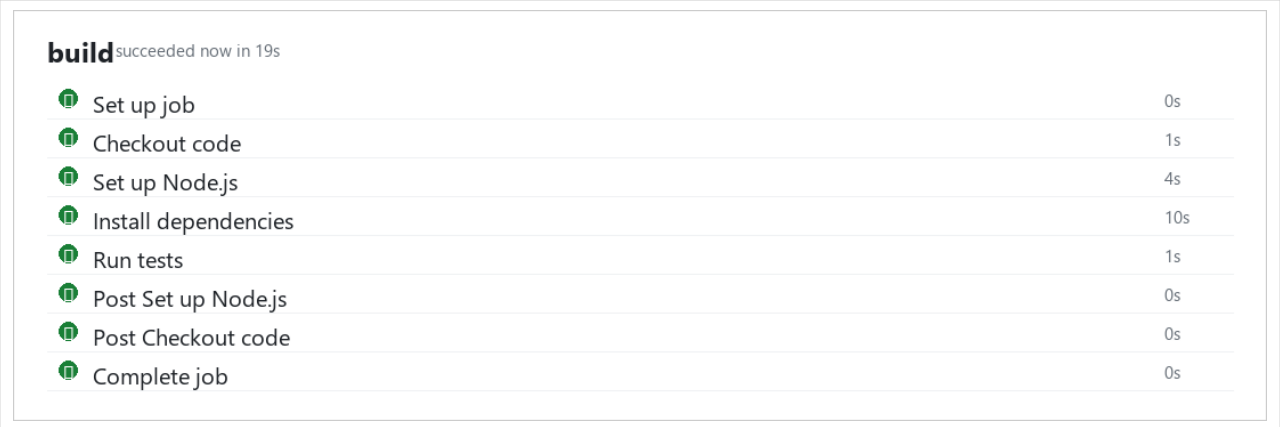


Fig 3: Successful workflow run with all steps passing (green checkmarks)

```
● Run tests
 1  Run npm test
 4
 5  > ci-actions-demo@1.0.0 test
 6  > jest
 7
 8  FAIL test/app.test.js
 9  ✕ adds two numbers correctly (4 ms)
10
11    ● adds two numbers correctly
12
13      expect(received).toBe(expected) // Object.is equality
14
15      Expected: 5
16      Received: -1
17
18        1 | const { add } = require('../src/app');
19        2 | test('adds two numbers correctly', () => {
20      > 3 |   expect(add(2, 3)).toBe(5);
21          |                     ^
22        4 | });
23
27  Tests:       1 failed, 1 total
31  Error: Process completed with exit code 1.
```

Fig 4: Failed workflow run after introducing an intentional error

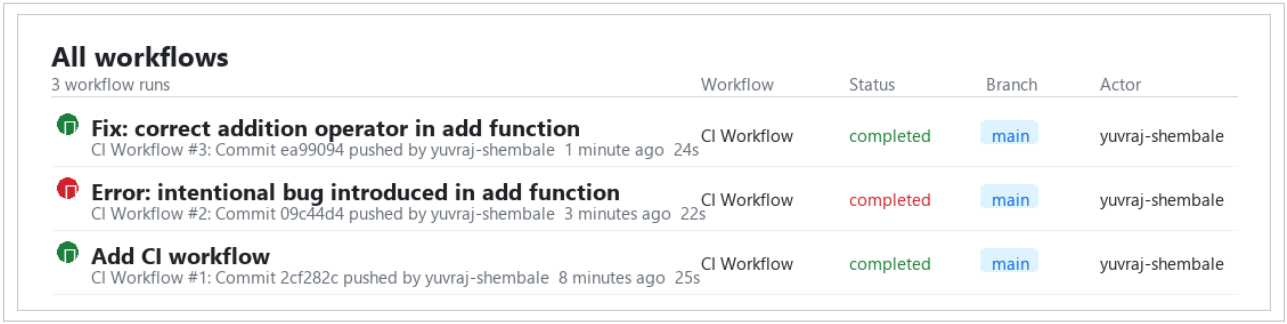


Fig 5: Successful workflow run after correcting the error

8. Best Practices

- Keep workflow files simple, modular, and easy to maintain.
- Use separate jobs for independent tasks such as build, test, and deploy.
- Run automated tests on every important code change, including pull requests.
- Pin or use well-maintained versions of actions and runtime environments (e.g., actions/checkout@v4).
- Never store passwords, API keys, or other secrets directly in workflow files — use GitHub Secrets instead.

9. Conclusion

GitHub Actions automates software validation and provides an important foundation for Continuous Integration directly within a GitHub repository. By configuring a workflow that automatically builds, tests, and reports on every push, development teams can catch integration issues early rather than discovering them later in the release cycle.

This assignment demonstrated the complete CI lifecycle: creating a workflow, observing a successful run, deliberately breaking the build to observe a failure, and then fixing it to restore a passing pipeline — reinforcing the practical value of automated CI in modern DevOps workflows.